PR12 | AI/ML BASED REAL-WORLD APPLICATION

Electricity Consumption and Bill Prediction System

Model development, evaluation, optimization and result analysis

Linear Regression and Ridge Regression with a React and Express application

Course project report

PR12 • CO1–CO5

Prepared 23 October 2026

The model estimates monthly household energy use in kWh. The application multiplies this estimate by a user-entered tariff to show an indicative bill.

Executive Summary

This project builds a small end-to-end household electricity estimator. A scikit-learn regression pipeline uses family size, rooms, appliance counts, area type, temperature and optional previous-month usage to estimate monthly consumption. A React interface calls an Express API; a Python bridge loads the fitted pipeline. SQLite stores each signed-in user's estimates separately.

The supplied Kaggle CSV contains 1,000 records and 19 columns. The model uses 12 predictors and Monthly_Usage_kWh as its continuous target. An 80:20 split leaves 800 rows for training and 200 for final testing. Five-fold cross-validation compares ordinary least-squares Linear Regression with Ridge candidates. Ridge alpha 1.0 wins by a very small margin.

Evaluation case	MAE	RMSE	R^2
Held-out test with all predictors	15.918 kWh	20.946 kWh	0.9656
Held-out test with prior usage hidden	36.039 kWh	45.613 kWh	0.8367

The full-feature R^2 is not a percent-accuracy claim. Prior-month use is strongly correlated with the target, and the no-history test is substantially harder.

Report guide

Pages	Sections
3–4	Problem definition, objectives and scope
5–8	Dataset, preprocessing and ML pipeline
9–11	Linear Regression, Ridge tuning and evaluation design
12–15	Metrics, results, errors and model coefficients
16–17	Web application architecture and workflow
18–19	Model considerations, reproducibility, CO mapping and sources

1 Problem Definition

Households often want a rough sense of next month's electricity requirement before a utility bill arrives. Appliance counts and household composition are informative, but they do not directly state monthly kilowatt-hours. This project frames the task as supervised regression: learn a mapping from household attributes to a continuous target, Monthly_Usage_kWh.

Problem statement

Given a household profile x with 12 available predictors, estimate y = Monthly_Usage_kWh. The model returns a non-negative monthly kWh estimate. The web application then computes an indicative bill as predicted kWh multiplied by the user's tariff per kWh.

Why regression

The target is numerical and ordered. A classifier would require artificial consumption bands and discard the size of the difference between estimates. Regression preserves the target's unit, so an absolute error can be discussed directly as kWh.

Project objectives

- Define a measurable target and household input set.
- Clean and split the dataset, fitting all preprocessing only on training data.
- Train Linear Regression as a baseline and Ridge as a regularized candidate.
- Tune Ridge with training-only five-fold CV and evaluate on a held-out test set.
- Analyze overall, subgroup, residual and cold-start errors.
- Connect the model to an authenticated web app and document limits.

The system presents a monthly estimate in kWh and computes a corresponding bill estimate from the tariff entered by the user.

2 Objectives and System Scope

The application turns the model into a household workflow. A new user registers with an empty history, enters home details and requests an estimate. Returning users sign in, inspect recent estimates and can begin with the latest estimate as an editable previous-month value.

Layer	Responsibility	Implementation
Frontend	Collect inputs; display kWh, bill and activity	React 19 dashboard
API	Validate, authenticate, predict, calculate bill	Express 5 JSON routes
Model bridge	Convert JSON to features and invoke model	Python, pandas and joblib
Estimator	Transform household data and predict kWh	scikit-learn Ridge pipeline
Persistence	Keep accounts, sessions and estimates	SQLite

What it returns

The result includes estimated usage, chosen tariff, estimated total and creation time. The arithmetic is transparent: bill estimate = predicted kWh × tariff per kWh.

3 Dataset Source and Selected Fields

The project uses the CSV associated with the Kaggle Electricity Bill Prediction page supplied for the project [1], stored as data/electricity_bill_prediction.csv. It has 1,000 rows and 19 columns.

Predictor group	Fields included
Home and people	Household_Size; Rooms; Area_Type
Appliances	AC_Units; Fan_Count; Light_Count; Refrigerator; TV_Count; Washing_Machine; Other_Appliances
Context and history	Previous_Month_Usage_kWh; Temperature_C
Target	Monthly_Usage_kWh

Fields excluded

Customer_ID is an identifier and could encourage memorization. Location is omitted because it is not needed for this compact model. Meter_Type and Tariff_Category do not directly describe consumption. Previous_Bill_BDT and Electricity_Bill_BDT are bill values, not the target; including current bill could leak information closely tied to use and tariff.

4 Dataset Profile and Data Quality

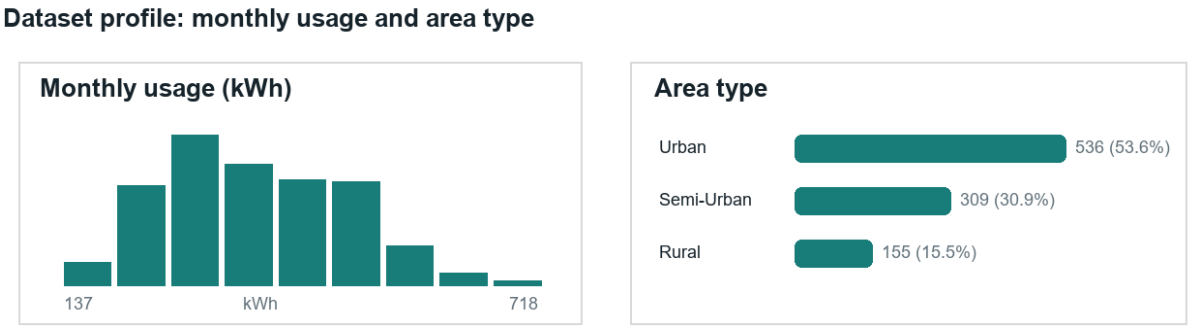


Figure 1. Monthly target distribution and area-type counts in the supplied CSV.

Profile item	Observed value
Rows / columns	1,000 / 19
Target mean / median / SD	370.78 / 357.55 / 108.90 kWh
Target range	136.8–718.0 kWh
Area type	Urban 536 (53.6%); Semi-Urban 309 (30.9%); Rural 155 (15.5%)
Exact duplicates removed	0
Missing values in required fields	0

The target centers near 371 kWh, with substantial variation across records. The area mix is imbalanced toward urban rows, so overall scores can hide weaker performance in smaller groups.

No required training field is missing in the current CSV. Imputation is still part of the deployed pipeline so blank optional inputs and future incomplete rows can be handled consistently.

5 Data Cleaning and Preprocessing

Preprocessing lives inside the same scikit-learn pipeline as the estimator. Each imputer, scaler and encoder is fitted on its training fold, then applied to validation or test rows. This prevents held-out rows from contributing preprocessing statistics.

Step	Implementation	Purpose
Schema	Strip column-name whitespace; check required fields	Catch incompatible CSV versions
Duplicates	Count and remove exact repeats	Avoid repeated rows as false independent evidence
Numeric conversion	Convert numeric features and target; malformed text becomes missing	Make values usable in regression
Target	Drop rows without Monthly_Usage_kWh	Training requires a known label
Numeric missing values	Median imputer	Simple fallback for incomplete rows
Scaling	StandardScaler on numeric features	Comparable scale for Ridge penalty
Area type	Mode imputer + one-hot; ignore unknown values	Encode category and tolerate unseen category

The required columns have zero missing values in the current file. The preprocessing steps still matter for a blank previous-month input. Pipeline and ColumnTransformer keep the same transformation logic in cross-validation, model fitting and inference [4].

6 End-to-End Machine-Learning Pipeline

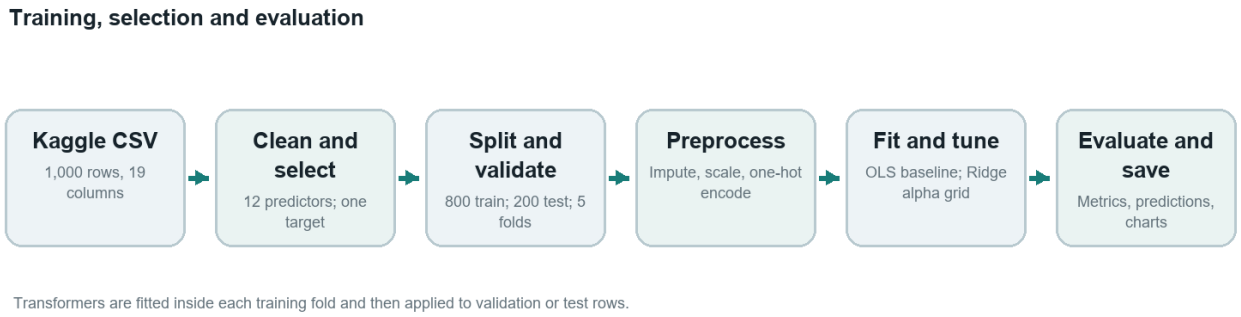


Figure 2. Workflow implemented by train_model.py.

The script selects 12 predictors and one continuous target, then splits once into training and held-out test subsets. Cross-validation and hyperparameter selection see only the training subset. The selected fitted pipeline is evaluated on the test rows once.

At inference, the saved pipeline performs the same imputation, scaling and encoding before prediction. Storing the preprocessing and estimator together avoids separately reimplementing feature preparation in Express.

Training outputs

The joblib artifact contains the fitted transformer and Ridge estimator. The results folder contains metrics.json, test_predictions.csv, cold_start_test_predictions.csv, error_by_usage_band.csv, model_coefficients.csv, actual_vs_predicted.png and residual_diagnostics.png.

7 Model Development with Linear Regression

Ordinary least-squares Linear Regression is the reference model. For a feature vector x, it estimates:

$$y_hat = \beta0 + \beta1x1 + \beta2x2 + ... + \beta pxp$$

The coefficients are selected to minimize squared residuals on the training data. This baseline is simple to explain and fast to train. It provides a reference point for testing whether regularization helps without changing the feature preparation or target.

Inputs and output

The model receives 11 numeric predictors and one categorical predictor, Area_Type. The target is Monthly_Usage_kWh. The estimator returns a continuous value; predict_usage.py clips negative estimates to zero because negative monthly use is not meaningful for the interface.

Interpretation and assumptions

A linear model represents additive relationships unless interaction or nonlinear terms are explicitly added. This version does not create appliance interactions, rated power, daily operating hours or nonlinear temperature effects. A coefficient is a conditional association under the fitted model, not proof that an appliance causes a specific metered increase.

LinearRegression minimizes residual sum of squares [2], making it more sensitive to very large misses than MAE. Reporting both MAE and RMSE helps describe typical and tail errors.

8 Model Optimization with Ridge Regression

Ridge adds an L2 penalty to squared error. Alpha controls its strength: a larger alpha shrinks coefficients more. Numeric predictors are standardized before the penalty is applied so measurement units do not dominate shrinkage.

$$\text{Objective} = \text{sum}((y - \text{y_hat})^2) + \alpha\ \text{sum}(\beta j^2)$$

GridSearchCV tests four alpha values with five-fold cross-validation on training rows. The scoring objective is MAE; the candidate with the lowest mean validation MAE is selected and refit on training data [3].

Ridge alpha	Mean CV MAE (kWh)	Fold SD (kWh)
0.1	16.7303	0.2017
1	16.7287	0.2041
10	16.7466	0.2311
100	17.8314	0.4139

Alpha 1.0 has the lowest CV MAE at 16.7287 kWh. Linear Regression's mean is 16.7305 kWh. The 0.0018 kWh gap is practically negligible. Ridge wins under the stated selection rule, but the experiment does not show a material advantage over the baseline.

9 Training and Test Protocol

Choice	Setting
Train / test	80:20; 800 training rows and 200 test rows
Random seed	91
Cross-validation	5 folds, shuffled, random_state 91
Search grid	Ridge alpha = 0.1, 1, 10, 100
Search score	Negative MAE, so a larger score means lower error
Selection	Choose lower mean training-CV MAE
Final metrics	Score held-out test after model selection

Why hold out test rows

The test set does not fit the imputer, scaler, encoder, coefficients or Ridge alpha. The 800 training records support cross-validation and candidate selection; the 200 test records provide a final evaluation on unseen rows. This separation follows the scikit-learn model-selection guidance [3].

Evaluation design

The fixed seed and recorded split make the training, model selection and held-out evaluation reproducible. Cross-validation scores are calculated on training folds, and the final metrics are calculated on the separate 200-row test set.

10 Performance Metrics

Metric	Formula	Interpretation
MAE	mean(|actual − predicted|)	Average absolute error, in kWh; lower is better.
RMSE	sqrt(mean((actual − predicted)²))	Penalizes large misses more heavily; lower is better.
R^2	1 − residual sum of squares / total sum of squares	Compared with predicting test mean; 1 is ideal, 0 is mean baseline, negative can be worse.

MAE is the tuning objective because it has a direct kWh interpretation and is less dominated by one large miss than RMSE. RMSE complements it by emphasizing the tail. R^2 is a dimensionless comparison with a constant-mean baseline; it is not a percent-accuracy score [2, 5].

Residual measures

The report also records mean and median signed error, the 90th-percentile and maximum absolute errors, and the percentages of held-out records within 10, 20 and 30 kWh. Residual is actual minus predicted: a positive value means underprediction; a negative value means overprediction.

These tolerance rates do not define universal acceptability. They summarize this held-out sample at chosen kWh thresholds. The relevant tolerance depends on meter resolution and intended use.

11 Held-Out Results and Model Comparison

Model	MAE (kWh)	RMSE (kWh)	R^2
Linear Regression	15.9257	20.9559	0.9655
Optimized Ridge	15.9184	20.9456	0.9656

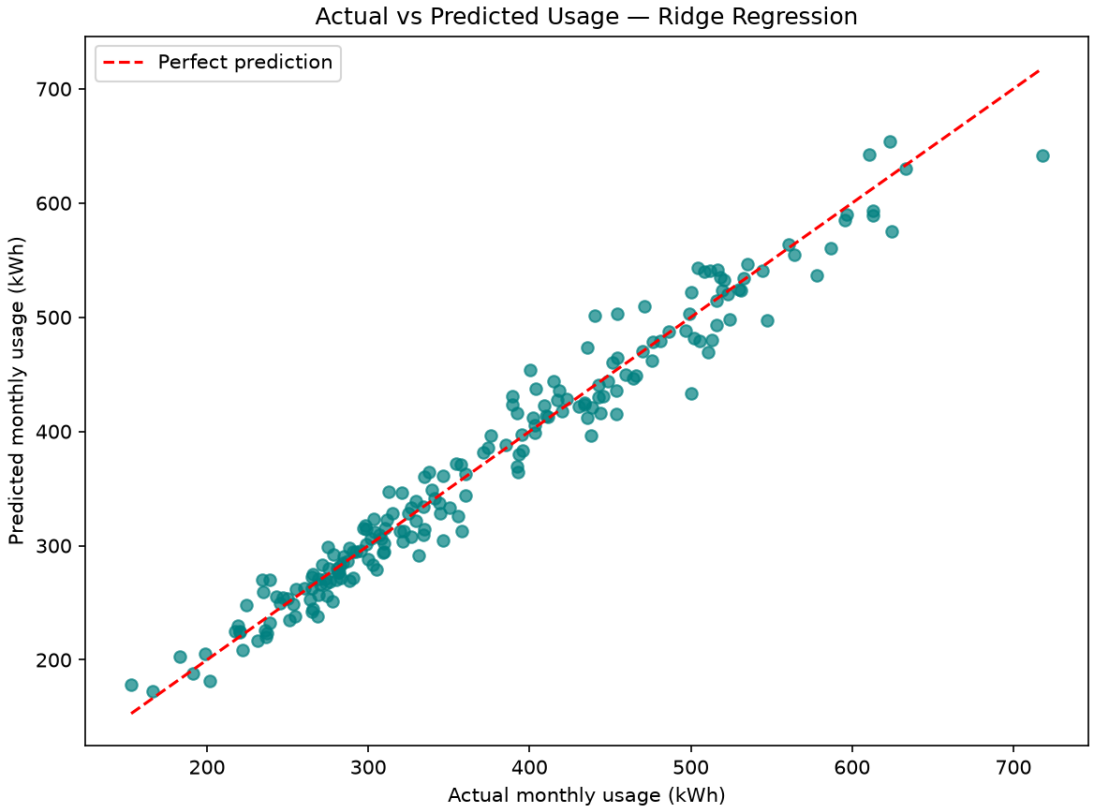


Figure 3. Held-out predictions plotted against observed values; the diagonal is perfect agreement.

Ridge lowers test MAE by 0.0073 kWh and RMSE by 0.0103 kWh compared with Linear Regression, relative reductions of about 0.05%. R^2 rises from 0.9655 to 0.9656. These results are practically indistinguishable on this split.

Selected Ridge metrics are MAE 15.918 kWh, RMSE 20.946 kWh and R^2 0.9656. Mean actual test usage is 371.746 kWh. The MAE is about 4.3% of this mean, but that ratio is not a calibrated per-household percentage error.

The scatter points cluster around the diagonal, indicating close agreement between observed and predicted usage on the held-out test set. The spread is wider among higher-usage records.

12 Residual Analysis and First-Visit Scenario

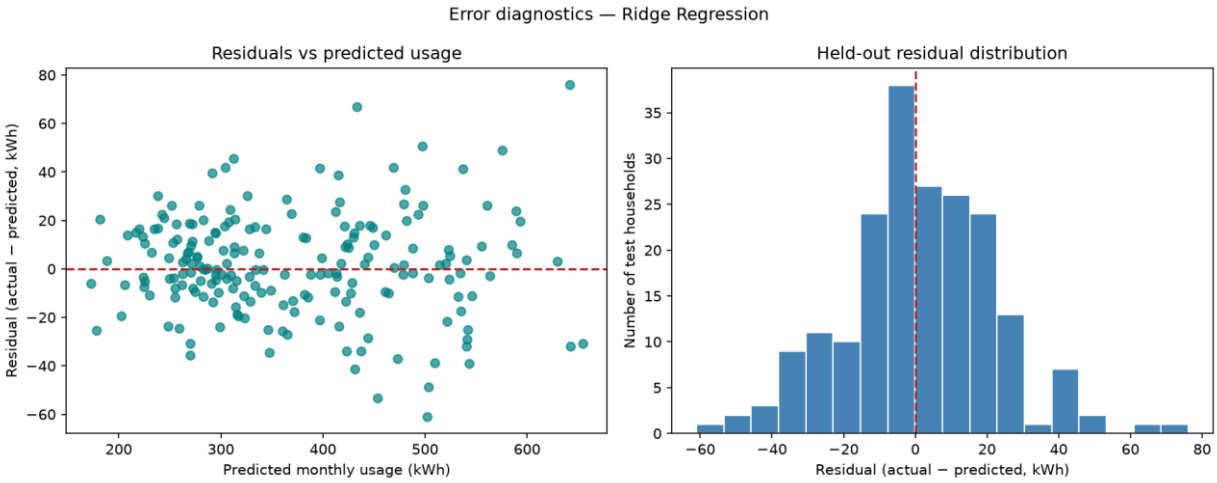


Figure 4. Residual = actual minus predicted; positive values indicate underprediction.

Usage quartile	Rows	Actual range (kWh)	MAE	Mean signed error
Q1	50	153.0–280.7	12.40	+0.09
Q2	50	281.6–344.1	11.88	+0.46
Q3	50	344.6–454.0	18.89	-0.37
Q4	50	454.1–718.0	20.50	+5.94

Across 200 rows, mean signed error is +1.533 kWh and median is +0.762 kWh. The 90th-percentile absolute error is 33.992 kWh; the maximum is 76.133 kWh. Estimates fall within 10, 20 and 30 kWh for 42.0%, 70.0% and 85.5% of test records.

The highest-use quartile has MAE 20.50 kWh versus 11.88–12.40 kWh in the first two quartiles. Its mean signed error is +5.94 kWh, showing average underprediction at the high end.

When previous-month usage is blank

When Previous_Month_Usage_kWh is hidden for every test row, the fitted training median is imputed. MAE rises from 15.918 to 36.039 kWh; RMSE becomes 45.613 kWh and R^2 falls to 0.8367. This stress case is more relevant to a new user who leaves the optional input blank.

13 Feature Contribution Analysis

Encoded feature	Coefficient
AC_Units	+43.00
Previous_Month_Usage_kWh	+38.40
Household_Size	+23.31
Other_Appliances	+10.67
Rooms	+7.83
TV_Count	+7.29
Light_Count	+4.90
Refrigerator	+4.52

The largest absolute numeric coefficients are AC_Units (+43.00), Previous_Month_Usage_kWh (+38.40) and Household_Size (+23.31). The pattern is consistent with previous consumption and cooling demand carrying substantial information about the current target.

Numeric coefficients correspond to standardized inputs, so their values describe a one-standard-deviation shift while other encoded values are held fixed. They are not kWh per additional appliance. The area categories use one-hot encoding; their coefficients should be read with the model intercept and coding structure.

Coefficient size is not a causal effect or guaranteed ranking in a different dataset. Correlated predictors share information; previous-month use is strongly associated with the target. A future explanatory version could add permutation importance or partial-dependence analysis.

Interpretability choice

Linear models keep the coursework implementation explainable. A tree-based model might capture interactions; it can be compared using the same cross-validation folds and metrics.

14 Full-Stack System Architecture

Application and prediction architecture

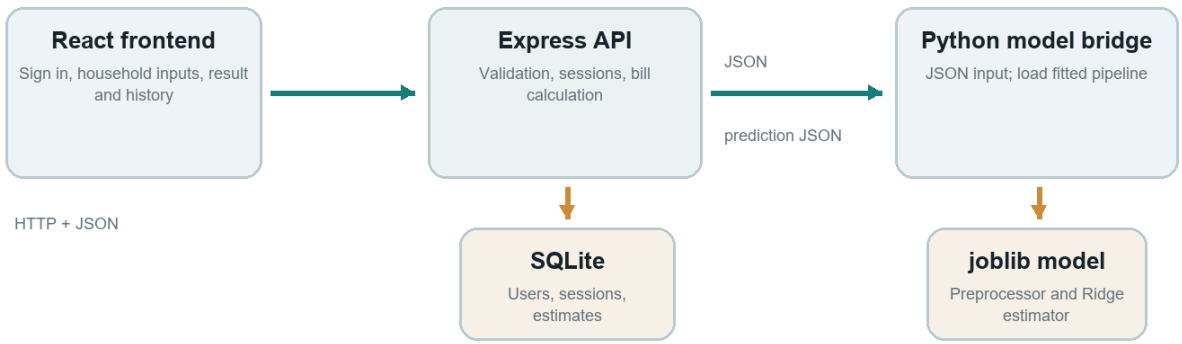


Figure 5. React calls Express; Express invokes Python and persists per-user data in SQLite.

The browser submits household input as JSON. Express checks the active session, validates numeric ranges and allowed area type, then invokes predict_usage.py. The Python process reads JSON from standard input, creates a one-row pandas DataFrame, loads electricity_usage_model.joblib and writes predicted kWh as JSON.

Express calculates estimated_bill = predicted_kwh × tariff_per_kwh, inserts inputs, prediction, tariff, bill and timestamp under the authenticated user ID, then returns the result. The frontend refreshes its recent-history panel.

Component	Responsibility
React + Vite	Single-page form and dashboard; Vite proxies API calls during development.
Express	Session routes, CSRF checks, validation, prediction coordination and bill arithmetic.
Python + scikit-learn	Load the fitted model artifact and return estimated kWh.
SQLite	Persist accounts, server-side sessions and user-linked predictions.

15 Application Workflow and Account History

API route	Purpose
GET /api/csrf-token	Issue token for state-changing requests.
GET /api/auth/me	Return signed-in user or null.
POST /api/auth/register	Validate credentials, hash password and create account/session.
POST /api/auth/login	Check credentials and establish session.
POST /api/auth/logout	Destroy current session.
GET /api/history	Return authenticated user's latest 20 estimates.
POST /api/predict	Validate inputs, run model, estimate bill and save record.

Passwords are hashed with bcryptjs before storage. Session data is server-side in SQLite; the cookie is HTTP-only, SameSite=Lax and lasts seven days, with the secure flag enabled in production. A per-session CSRF token protects state-changing requests. History reads are filtered by the current user's ID.

A new account starts with empty history. Returning users see up to 20 recent estimates. The latest predicted kWh may be prefilled as previous-month usage, but it is only an estimate; the UI tells users to replace it with a meter reading when they have one. Reusing predictions can reinforce earlier model error.

This local authentication setup is for a coursework demonstration. Public deployment needs a strong secret, HTTPS, authentication rate limits, monitoring, backups, data-retention choices and a production security review.

16 Model Considerations and Improvements

Model consideration	Why it matters	Next step
Prior usage is powerful	Full-feature score exceeds the no-history score.	Report cold-start separately; test feature ablations.
Linear form	No nonlinear or interaction effects.	Compare one tree/boosting model using the same CV folds.
Prediction may prefill prior input	A prior estimate is not a meter reading.	Label source or default to blank.

Recommended evaluation sequence

Compare an appliance-only model, a model with previous-month use and a no-history scenario. Compare Ridge with one nonlinear model using the same cross-validation folds and MAE/RMSE/R^2, then review errors by area and consumption band.

The current optimization demonstrates model selection; Ridge improves the test MAE by a small amount. A useful next step is to study which available predictors contribute most to first-visit estimates.

17 Reproducibility, Course Mapping and References

Reproduce the model

From the project folder, install requirements and run python train_model.py. The script writes models/electricity_usage_model.joblib and metrics/diagnostics into results/. Express invokes the saved pipeline through predict_usage.py.

Training environment for this report run: Python 3.14.3, scikit-learn 1.8.0, NumPy 2.4.3, pandas 3.0.1, matplotlib 3.11.1 and joblib 1.5.3. The random seed is 91. The CSV is included in the project for a repeatable local run.

Syllabus outcome	Project evidence
CO1 — Problem formulation	Continuous target, input fields, scope and use case.
CO2 — Data preparation	Schema validation, duplicate removal, conversion, imputation, scaling and encoding.
CO3 — Model development	Linear Regression baseline and Ridge inside a reusable pipeline.
CO4 — Evaluation and optimization	80:20 holdout, five-fold CV, alpha search, MAE, RMSE, R^2 and diagnostics.
CO5 — Application and communication	React + Express + Python + SQLite integration, account history and report.

References

[1] Kaggle, Electricity Bill Prediction dataset page supplied for this project: https://www.kaggle.com/datasets/armanahmedtaief/electricity-bill-prediction
[2] scikit-learn, LinearRegression API: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html
[3] scikit-learn, hyperparameter tuning and GridSearchCV: https://scikit-learn.org/stable/modules/grid_search.html
[4] scikit-learn, Pipeline API: https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html
[5] scikit-learn, regression metrics: https://scikit-learn.org/stable/api/sklearn.metrics.html

The project demonstrates a full regression workflow and connected web interface. Its reported metrics summarize the selected test split, while the cold-start results show how performance changes when previous-month usage is unavailable.